Action-Conditioned Predictive Diagnostics for JEPA World Models

Author names to be supplied for arXiv v1

July 2026

Abstract

Latent predictive world models avoid reconstructing pixels, but latent prediction alone does not show whether a state-preserving visual change—noise, blur, degraded resolution—remains harmless once the model rolls the state forward and a planner acts on the prediction. We introduce Action-Conditioned Predictive Consistency (ACPC), a diagnostic for frozen checkpoints: encode a clean and a visually perturbed view of the same history, roll both forward under the same actions, and measure how far the two predicted latent trajectories separate. Exact samplewise inequalities tie this separation to the resulting change in multi-step prediction error and to the movement of planner candidate costs. Because a collapsed representation would make the separation vanish, we pair the checkpoint-level radius (ATR) with a separation test on pairs whose logged task states differ (SMPR). On LeWM across four control tasks, eight-step ACPC under the recorded actions predicts perturbation-induced error changes with $51.3 \pm 3.5\%$ lower cross-validated MAE than the strongest of three same-horizon controls whose action information is destroyed, so the signal comes from the recorded actions rather than from rollout length. ACPC also improves leave-one-task-out prediction of CEM cost movement and decision regret; ATR–SMPR thresholds calibrated on source tasks screen checkpoints from held-out tasks, order checkpoint pairs correctly under blur and resize at the tested severities, and port to PLDM after model-family recalibration.

Keywords: visual world models; predictive stability; action-conditioned diagnostics; calibration transfer; visual corruption.

Code and data release: `https://github.com/Anguo-star/lewm-acpc-diagnostics`

1 Introduction

World models support control by predicting how candidate actions change the environment [11, 12]. Joint-embedding predictive architectures (JEPAs) move this prediction problem from pixels to learned representations [16, 1, 3]. By avoiding pixel reconstruction, they can reduce pressure to model nuisance detail [28, 17]. This is appealing for visual control, where lighting, texture, or image quality may change even when the physical situation does not. Latent prediction alone, however, does not establish that a trained world model will produce stable action-conditioned predictions under such changes.

For control, whether a visual difference is irrelevant depends on the state and the actions being considered. A background texture can be ignored, whereas a contact pose, doorway state, or object–goal relation may determine which action is useful. Encoder invariance cannot resolve this distinction on its own because a planner does not act directly on the encoder output. It rolls the representation forward under candidate actions and ranks the resulting predictions with a cost. A small encoder discrepancy may grow through this computation, whereas a larger one may contract before it affects the cost. This motivates the central question: after the same action sequence is applied, do nominal and perturbed views of one trajectory still produce similar predicted futures?

Low rollout disagreement alone is not enough. A model that maps every observation to a constant representation would make all paired rollouts identical while retaining no state information for control. A useful diagnostic must therefore test two properties together: same-state visual views should remain close throughout the predicted rollout, while pairs identified as different by task state coordinates should remain separated. The latter test is a deliberately limited collapse guard. It assesses only the supplied coordinate labels and does not certify every semantic or action-relevant distinction.

We introduce Action-Conditioned Predictive Consistency (ACPC) to quantify same-state rollout stability. ACPC propagates the nominal and visually perturbed histories of one trajectory under a shared action sequence

and measures the distance between the two predicted latent trajectories; computing the score requires neither a realized future nor a task outcome. Two exact inequalities make the score interpretable: evaluating both predictions against the same logged future bounds how much the perturbation can change the prediction error, and, for LeWM's squared latent goal cost, a second bound limits how far each candidate's cost can move and yields fixed-pool stability certificates from the clean candidate margins.

A small rollout radius alone is still not sufficient, so we summarize checkpoints with two complementary numbers. Action-Conditioned Trajectory Radius (ATR) reports the high-quantile normalized disagreement over logged evaluation windows, while Selective Margin Pass Rate (SMPR) checks that pairs labeled as different states by task coordinates remain outside the same-state radius by a margin. A calibrated score combining ATR and SMPR screens frozen checkpoints for visual predictive sensitivity. ACPC is an evaluation tool, not a new training objective.

We evaluate this diagnostic on LeWM [19] across TwoRoom, PushT, Reacher, and Cube, using Gaussian observation noise for calibration. On the unaugmented checkpoints, eight-step ACPC under the recorded actions predicts the perturbation-induced change in prediction error about 51% better, in cross-validated MAE, than the strongest control with the same rollout length but destroyed action information (Section 4.3). ACPC computed along planner candidates consistently improves prediction of CEM decision regret (Section 4.4). Thresholds selected on source tasks screen checkpoints from held-out tasks with balanced accuracy 0.86–0.90 across all cross-task calibration splits (Section 4.5), and the calibrated score orders checkpoint pairs correctly under blur and resize at the tested severities (Section 4.7). The same analysis applies to PLDM after model-family recalibration (Section 4.6).

The contributions are:

1. **A paired-rollout diagnostic** (Section 3). ACPC measures how same-state visual changes propagate through action-conditioned predictions; ATR summarizes the checkpoint-level radius, and SMPR guards against constant-representation collapse.
2. **Exact links to prediction and planning quantities** (Propositions 1 and 2). Samplewise inequalities connect rollout disagreement to common-future error drift and to squared latent goal-cost movement.
3. **Controlled evidence beyond the calibration tasks** (Section 4). Experiments show that ACPC predicts perturbation-induced error changes and CEM decision regret, and that calibrated thresholds transfer to held-out tasks, to blur and resize, and to a second model family.

2 Related Work

Prior work addresses three complementary parts of this problem: learning latent predictive representations, preserving control-relevant distinctions under visual shifts, and evaluating learned dynamics.

Latent predictive world models. DreamerV3 and TD-MPC2 illustrate how learned dynamics can support control by predicting the consequences of candidate actions [11, 12]. JEPAs replace pixel reconstruction with prediction in representation space, first for images and then for video [16, 1, 3, 2]. LeWM instantiates this idea as an end-to-end JEPA world model stabilized by SIGReg [19, 7], while PLDM provides a different joint-embedding dynamics design [25, 26, 18]. Analyses of latent prediction explain why it can suppress nuisance or noisy features [28, 17, 14]; Seq-JEPA studies the related balance between invariant and equivariant prediction [9]. These results motivate latent predictive learning, but our focus is checkpoint evaluation: whether the discrepancy between two visual views of the same trajectory contracts or grows as the predictor rolls them forward under shared actions.

Robust visual control and selective invariance. DrQ, DrQ-v2, and SODA improve visual control through training-time augmentation [15, 32, 13]. ViGMO and VIBR learn invariances in latent or value space [20, 6], whereas ReOI modifies observations at test time to reduce distractor sensitivity in visual model-predictive control [5]. These methods seek to train or repair a more robust controller. We instead ask how to audit visual sensitivity in an already trained predictive model.

Control-aware representation learning also explains why robustness cannot mean removing every difference. Bisimulation-based methods preserve distinctions in rewards and transitions [8, 33, 24, 27], while value-equivalent and value-aware models preserve distinctions that matter to downstream value or planning [10, 29]. This motivates the two-sided structure of our diagnostic: same-state visual views should agree after prediction, but

low disagreement is considered meaningful only when pairs marked as different by task state coordinates remain separated. Our state-coordinate test is narrower than a learned bisimulation or semantic criterion; its role is to expose the constant-representation failure mode.

Diagnostics of learned dynamics. Recent work audits whether learned dynamics retain action or temporal structure using inverse prediction, latent action decoding, cycle consistency, group-action tests, or compatibility with generated futures [4, 34, 23, 21, 30]. Related approaches place action-conditioned consistency inside the training objective [31] or diagnose kinematic failures in long-horizon rollouts [22]. Together, these studies motivate evaluating the predictor directly rather than inferring its behavior from encoder geometry alone. ACPC adds a complementary paired-view test: under one shared action sequence, it measures how much a state-preserving visual change moves the predicted rollout. We then test how that displacement relates to the change in prediction error against a common observed future, to fixed-pool candidate-cost movement, and to adaptive-CEM decision regret. For checkpoint screening, ATR and SMPR combine the rollout radius with the limited state-coordinate proxy guard; their numerical calibration is performed within each model family.

3 Action-Conditioned Predictive Consistency as a Diagnostic

A visual perturbation can change pixels without changing the underlying control state. It nevertheless displaces the encoded history, and learned dynamics can amplify, rotate, or contract that displacement before a planner reads out a cost. Encoder invariance alone therefore leaves two questions unanswered: how far do the predicted futures separate under the same actions, and can that separation change a planning decision?

The desired diagnostic has two sides. Two visual views of the same state should remain inside a small action-conditioned rollout tube. At the same time, states that differ under a declared task proxy should remain outside that tube. The first side supplies a predictive radius; the second prevents a low radius from being mistaken for robustness when the representation has collapsed. We first define the radius and connect it to prediction error and planner decisions, then introduce the complementary task-proxy margin and checkpoint score.

3.1 Paired rollout radius

Let h be a clean observation–action history—the input the checkpoint's encoder consumes—and let $\tilde{h}$ be a visually perturbed view of the same underlying state trajectory. A frozen encoder gives $z = E_\theta(h)$ and $\tilde{z} = E_\theta(\tilde{h})$. Both branches receive the same action sequence $\mathbf{a} = (a_0, \dots, a_{H-1})$. For its length-$k$ prefix, define

$$\hat{z}_k = F_\theta^k(z, \mathbf{a}_{0:k-1}), \qquad \hat{\tilde{z}}_k = F_\theta^k(\tilde{z}, \mathbf{a}_{0:k-1}). \tag{1}$$

Here F_θ is the action-conditioned predictor and H is the number of autoregressive future steps. The planner does not read raw predictor outputs: it scores candidates in a normalized projection of the embedding space. The map Π denotes that projection, so every distance below lives in the same space in which planner costs are computed; our implementation stores these projected embeddings directly, and Π then requires no extra computation. With nonnegative weights $\sum_{k=1}^{H} \alpha_k = 1$, stack the whole rollout as

$$\bar{G}_{\mathbf{a}}(z) = \left[\sqrt{\alpha_1}\Pi(\hat{z}_1)^\top \quad \cdots \quad \sqrt{\alpha_H}\Pi(\hat{z}_H)^\top\right]^\top. \tag{2}$$

Action-Conditioned Predictive Consistency (ACPC) is the distance between the two weighted predicted rollouts:

$$\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}) = \left\|\bar{G}_{\mathbf{a}}(E_\theta(h)) - \bar{G}_{\mathbf{a}}(E_\theta(\tilde{h}))\right\|_2 = \left(\sum_{k=1}^{H} \alpha_k \left\|\Pi(\hat{z}_k) - \Pi(\hat{\tilde{z}}_k)\right\|_2^2\right)^{1/2}. \tag{3}$$

In plain terms, ACPC asks how differently the model predicts the future when only the visual appearance of the observed history changes. Encoder shift $\|z - \tilde{z}\|_2$ measures the upstream displacement before prediction; ACPC measures what remains after both histories are propagated under the same actions. Sharing actions isolates visual sensitivity under a fixed intervention; it does not by itself establish correct responses to other actions. Unless stated otherwise we use uniform weights $\alpha_k = 1/H$ and the fixed protocol horizon $H = 8$, the longest horizon for which every logged evaluation window contains a complete observed future (Appendix B). The planner analyses in Section 4.4 instead use $H = 5$, matching the five-action planning horizon of the evaluated CEM configuration.

3.2 Common-future error drift

The rollout radius has a direct consequence for prediction error. Let $Y^H_{\mathbf{a}}$ be one actually observed future under the same recorded action sequence, projected, stacked, and weighted exactly as in Equation (2). This target is not produced by either prediction branch and is used only for evaluation. Comparing both branches with this one future isolates the error change caused by the visual perturbation.

Proposition 1 (Common-future error-drift bound)**.** *Define*

$$e_h = \|\bar{G}_{\mathbf{a}}(E_\theta(h)) - Y^H_{\mathbf{a}}\|_2, \qquad e_{\tilde{h}} = \|\bar{G}_{\mathbf{a}}(E_\theta(\tilde{h})) - Y^H_{\mathbf{a}}\|_2.$$

Then, for every paired sample,

$$|e_{\tilde{h}} - e_h| \le \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}). \tag{4}$$

The adverse increase $(e_{\tilde{h}} - e_h)_+$, where $(x)_+ = \max(x, 0)$, obeys the same bound.

Proof. Apply the reverse triangle inequality in the weighted projected-rollout space. Both errors must use the same $Y^H_{\mathbf{a}}$. □

This proposition controls a change in error, not prediction accuracy. A small ACPC value can coexist with two inaccurate predictions; it says only that the two errors to the common future cannot differ by more than the paired-rollout radius. ACPC itself does not require the realized future. The prediction experiment in Section 4.3 uses that future only to evaluate whether ACPC is informative about the bounded quantity, which we call the *error drift* $d = |e_{\tilde{h}} - e_h|$.

3.3 Candidate-cost drift and planner stability

The planner link is a radius–margin argument: the perturbation can move each candidate's cost by at most a bounded amount, so a fixed-pool decision can change only when some cost moves far enough to cross the gap that separated the clean candidates. For candidate action sequence $\mathbf{a}^j$, let

$$x_j = \Pi(F^H_\theta(E_\theta(h), \mathbf{a}^j)), \qquad \tilde{x}_j = \Pi(F^H_\theta(E_\theta(\tilde{h}), \mathbf{a}^j))$$

be the final clean and perturbed predicted representations. Their displacement $r_j = \|x_j - \tilde{x}_j\|_2$ is the final-step component of the candidate-specific $\mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)$. If $\alpha_H > 0$, then $r_j \le \mathrm{ACPC}_H(h, \tilde{h}, \mathbf{a}^j)/\sqrt{\alpha_H}$; the planner experiments in Section 4.4 record r_j directly.

Proposition 2 (Squared goal-cost drift and fixed-pool stability)**.** *Suppose the planner scores candidate j against fixed goal embedding g by $C_j = \|x_j - g\|_2^2$ and $\tilde{C}_j = \|\tilde{x}_j - g\|_2^2$. Define*

$$b_j = r_j\big(\|x_j - g\|_2 + \|\tilde{x}_j - g\|_2\big). \tag{5}$$

Then every candidate satisfies $|\tilde{C}_j - C_j| \le b_j$.

Let w and $\tilde{w}$ be the clean and perturbed winners of the same pool. The clean-history decision regret obeys

$$0 \le C_{\tilde{w}} - C_w \le b_{\tilde{w}} + b_w. \tag{6}$$

If the clean winner is unique and

$$\min_{j \ne w}\big(C_j - C_w - b_j - b_w\big) > 0, \tag{7}$$

then the perturbed branch has the same winner. If $\mathcal{E}$ is the clean top-k elite set and

$$\min_{i \in \mathcal{E},\, j \notin \mathcal{E}}\big(C_j - C_i - b_j - b_i\big) > 0, \tag{8}$$

then the perturbed branch has exactly the same elite set.

The quantity b_j is a candidate-specific upper bound on the movement of the squared goal cost. The regret inequality bounds the extra clean-history model cost of choosing the perturbed winner. The last two conditions subtract the allowed movements from each clean candidate gap; a positive remainder means that the gap cannot be crossed. These statements use the evaluated endpoints, not a global Lipschitz approximation. Proofs

and equality cases are in Appendix A. Because every candidate must be evaluated under both histories, these certificates support a post-hoc audit rather than a cheap online pre-screen.

CEM repeatedly fits its next proposal to the current elite set. With common random numbers, identical proposals generate the same ordered candidate pool, and an identical certified elite set produces the same next proposal.

Corollary 1 (Conditional CEM stability)**.** *Consider clean and perturbed CEM runs initialized by the same proposal and sampled with common random numbers. While their ordered candidate pools remain aligned, if Equation* (8) *holds at the current iteration, both runs fit the same next proposal. If it holds at every iteration, their pools, proposals, and final actions remain aligned by induction.*

The guarantee stops at the first iteration for which the elite condition fails. It concerns candidate selection and model cost under paired planner evaluations; it is not a statement about simulator return. Appendix G specifies the fixed-pool and adaptive-CEM evaluation contract.

3.4 Why low radius needs a task-proxy margin

The two propositions explain why a small same-state radius is useful, but not why it is sufficient. A constant representation makes every ACPC value zero and destroys all state distinctions. We therefore pair the radius with an explicit separation test on declared state-coordinate proxies.

First convert pairwise ACPC to a checkpoint-level radius. Evaluation uses a fixed set of *anchors*: an anchor is one logged history window together with its H-step observed future and recorded actions. Throughout this section, Q_q denotes the empirical q-quantile, n is the number of anchors, and M is the number of perturbation draws per anchor. For anchor i, let $u^{\mathrm{obs}}_{i,0}$ be the embedding of the last clean history observation in the projected space selected by Π, and let $u^{\mathrm{obs}}_{i,1:H}$ be the embeddings of the next H actually observed clean frames in that same space. Define the clean-motion scale

$$s_i = Q_{0.50}\Big(\big\{\|u^{\mathrm{obs}}_{i,k} - u^{\mathrm{obs}}_{i,k-1}\|_2\big\}_{k=1}^{H}\Big). \tag{9}$$

This anchor-specific scale expresses rollout distances in units of the anchor's ordinary clean motion; it uses no task outcome. With a small numerical stabilizer $\varepsilon_{\mathrm{norm}}$ (value in Appendix B) and perturbation draw m, define

$$R_i^{(m)} = \frac{\mathrm{ACPC}_H(h_i, \tilde{h}_i^{(m)}, \mathbf{a}_i)}{s_i + \varepsilon_{\mathrm{norm}}}, \qquad \bar{R}_i = \frac{1}{M}\sum_{m=1}^{M} R_i^{(m)}. \tag{10}$$

The raw *Action-Conditioned Trajectory Radius* (ATR) is

$$\mathrm{ATR}_q^{\mathrm{raw}}(\theta) = Q_q\big(\{\bar{R}_i\}_{i=1}^{n}\big). \tag{11}$$

ATR is one number per checkpoint: it reports the upper-quantile radius of the same-state rollout tube. We use q90 because a mean can hide a small set of high-radius anchors; checkpoint-level conclusions are stable across horizons $H \in \{1, 2, 4, 8\}$ and quantiles q80–q95 (Table 4). This is a descriptive summary, not a candidate-distribution tail guarantee.

For the complementary test, let y_i be the standardized logged task-state vector at the end of anchor i's observed H-step future, and let $\ell(y_i)$ be a declared task-specific coordinate label. Let $d_{0.35}$ be the q35 of all off-diagonal Euclidean distances among the standardized endpoint states; this neighborhood radius and the margin below are protocol constants fixed a priori. Each eligible anchor chooses the nearest neighbor $j(i)$ with a different label and distance at most $d_{0.35}$. The predictor rolls both clean histories under the anchor actions $\mathbf{a}_i$; for the neighbor these are not the actions it actually executed, but reusing the anchor's actions keeps a single shared action sequence on both sides. Their normalized proxy-different distance is

$$D_i^{\mathrm{diff}} = \frac{\big\|\bar{G}_{\mathbf{a}_i}(E_\theta(h_i)) - \bar{G}_{\mathbf{a}_i}(E_\theta(h_{j(i)}))\big\|_2}{s_i + \varepsilon_{\mathrm{norm}}}. \tag{12}$$

Using the same rollout metric, anchor action sequence, and anchor scale on both sides makes the comparison direct. The *Selective Margin Pass Rate* (SMPR) is

$$\mathrm{SMPR}_{q,\delta}(\theta) = \frac{1}{|\mathcal{I}|}\sum_{i\in\mathcal{I}} \mathbf{1}\big[D_i^{\mathrm{diff}} > \mathrm{ATR}_q^{\mathrm{raw}}(\theta) + \delta\big], \tag{13}$$

where $\mathcal{I}$ contains anchors with an eligible label-crossing neighbor. The reported protocol fixes $q = 0.90$ and normalized margin $\delta = 0.10$. SMPR is the fraction of tested proxy-different pairs that remain outside the same-state tube by more than that margin.

ATR and SMPR must be read together. A constant representation makes both ATR and every D_i^{diff} zero, so it fails the SMPR test. Conversely, a model can separate the tested proxy classes while remaining visually sensitive, which yields a large ATR. These proxies test only the declared coordinate distinctions; they do not prove semantic validity, action relevance, or correct behavior. Appendix B gives the exact coordinate rules and eligible-pair counts.

3.5 Checkpoint-level calibration

The raw ATR in Equation (11) defines the tube used by SMPR. To compare augmentation levels across tasks, the experiments also use a task-relative ATR. Let θ_0 be the corresponding unaugmented checkpoint for the same task, training run, and model family (its raw ATR is strictly positive in every evaluated setting, so the ratio is well defined), and set

$$\widetilde{\mathrm{ATR}}_q(\theta;\theta_0) = \frac{\mathrm{ATR}_q^{\mathrm{raw}}(\theta)}{\mathrm{ATR}_q^{\mathrm{raw}}(\theta_0)}. \tag{14}$$

The raw quantity remains inside Equation (13); the relative quantity equals one at the reference checkpoint and is used for cross-checkpoint calibration.

For model family $\mathcal{F}$ and visual shift υ (Gaussian noise, blur, or resize), thresholds t_R and t_M combine the two sides; every quantity entering the score is measured under that shift. With a small constant $\varepsilon_{\text{score}}$ (value in Appendix B), define

$$S_{\mathcal{F},\upsilon}(\theta;\theta_0) = \min\left\{ \frac{t_R - \widetilde{\mathrm{ATR}}_q(\theta;\theta_0)}{|t_R| + \varepsilon_{\text{score}}}, \frac{\mathrm{SMPR}_{q,\delta}(\theta) - t_M}{|t_M| + \varepsilon_{\text{score}}} \right\}. \tag{15}$$

Thus $S \geq 0$ exactly when both weak inequalities pass: relative ATR is at most t_R and SMPR is at least t_M. Computing the score for a fixed checkpoint uses no task outcome. Planning success rates enter only when the thresholds are selected on source tasks and evaluated on other tasks. Thresholds are selected separately within each model family; the equations do not imply shared raw thresholds across architectures.

For a comparison with reference checkpoint θ_0, define

$$\Delta S_{\mathcal{F},\upsilon}(\theta_1;\theta_0) = S_{\mathcal{F},\upsilon}(\theta_1;\theta_0) - S_{\mathcal{F},\upsilon}(\theta_0;\theta_0). \tag{16}$$

A positive ΔS means only that θ_1 improves on its reference under the same calibrated scoring map. It does not assign an absolute robustness label to either checkpoint.

Table 1 summarizes the chain. The prediction experiment (Section 4.3) evaluates whether ACPC is informative about the error drift bounded by Proposition 1; the CEM experiment (Section 4.4) analyzes the candidate-cost and selection quantities in Proposition 2 and corollary 1; and the cross-task, PLDM, and blur/resize analyses (Sections 4.5 to 4.7) evaluate the calibrated ATR–SMPR score. These are related but distinct empirical claims.

Table 1: Reader guide to the ACPC diagnostic chain.

Object	Plain-language meaning	Empirical role
ACPC	Distance between the clean and perturbed predicted futures of one history under shared actions.	Prediction and planner analyses
b_j	Upper bound on how far candidate j's squared goal cost can move.	Fixed-pool planner audit
$\mathrm{ATR}^{\mathrm{raw}}$	q90 radius of the normalized same-state rollout tube.	Tube used by SMPR
D_i^{diff}	Rollout distance to a nearby state with a different coordinate-proxy label.	Proxy-separation test
SMPR	Fraction of proxy-different pairs that stay outside the tube by the margin.	Collapse guard
$\widetilde{\mathrm{ATR}}$, S	Task-relative radius; the smaller (worse) of the two calibrated margins.	Sweep and cross-task screening
ΔS	Change in the calibrated score relative to a reference checkpoint.	Blur/resize ordering

4 Experiments

Figure 1 provides an overview of planning success and the two checkpoint-level diagnostics across the Gaussian sweep. We first state the common evaluation protocol, then examine prediction, planning, and transfer claims

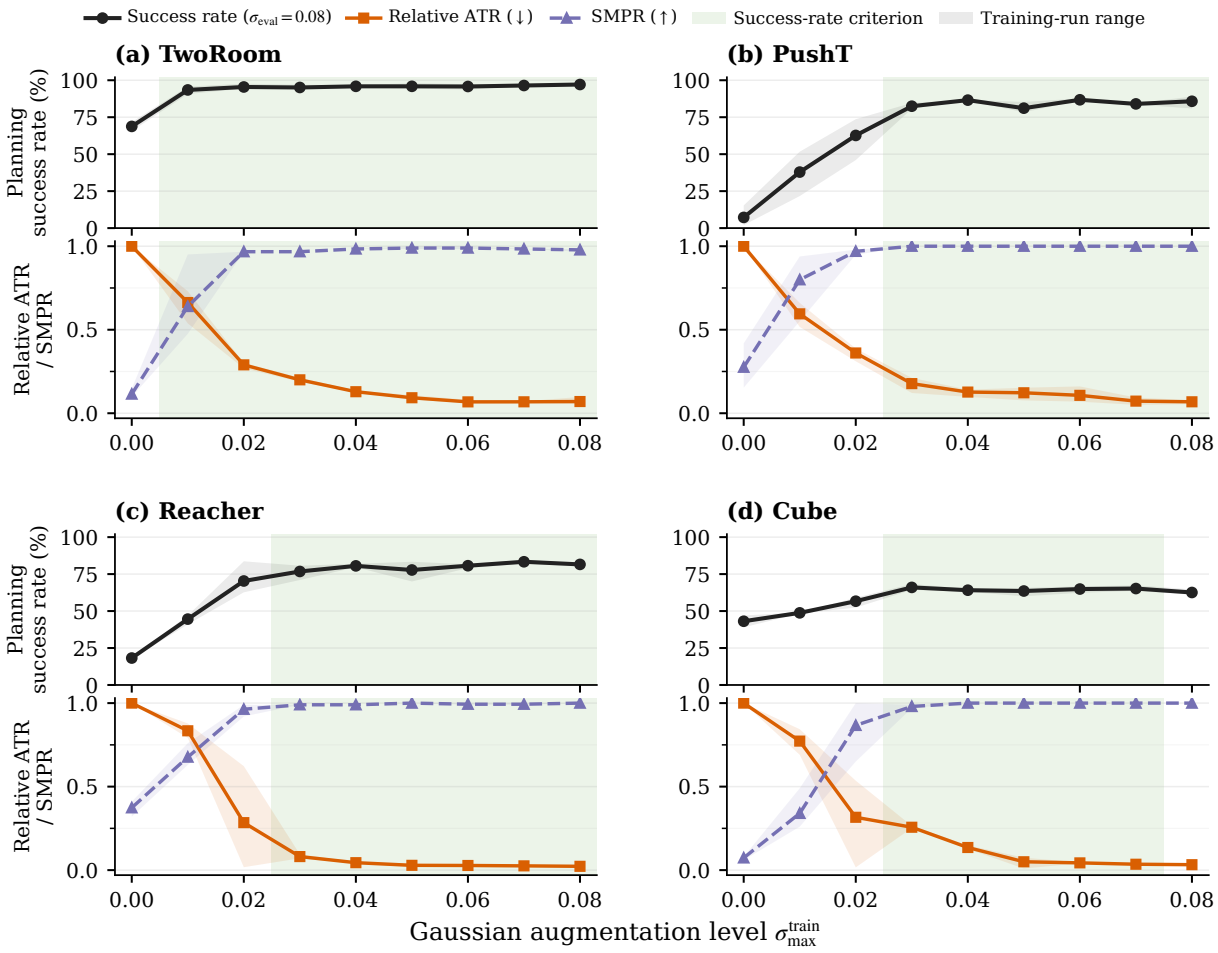

Figure 1: Planning success rate (evaluation noise $\sigma = 0.08$), relative ATR, and SMPR across the Gaussian-augmentation sweep. Lines are across-run means; shaded ranges span the three run means; green bands mark levels meeting the success-rate criterion (Section 4.1) in a majority of runs. Lower ATR (relative to the unaugmented checkpoint) and higher SMPR are favorable.

separately.

4.1 Evaluation setup

Models, tasks, and planning evaluation. LeWM [19] is the primary model family; PLDM [25, 26] provides a second joint-embedding dynamics architecture. We evaluate TwoRoom navigation, PushT planar manipulation, Reacher arm control, and OGBench-Cube (Cube) 3D manipulation. We follow the standard LeWM latent-space MPC evaluation: CEM optimizes five-action candidate sequences with the frozen world model (plan horizon 5), and performance is the percentage of evaluation episodes that reach the task goal, reported as *planning success rate*. Throughout, a *visual perturbation* is one sampled change to the observations of one history, and a *visual shift* is the corresponding distribution (Gaussian noise, blur, or resize). The main evaluation adds Gaussian noise $\sigma = 0.08$ to observations while keeping the goal image clean; additional evaluations use Gaussian blur ($k = 15$) and resize (scale 0.25).

For each model family, the Gaussian-augmentation sweep contains nine checkpoints per task: the *unaugmented checkpoint* (trained without noise augmentation) and checkpoints trained with full-sequence noise $\sigma_{\max} \in \{0.01, \ldots, 0.08\}$. Each LeWM task–augmentation setting is trained three times (*training runs*, seeds 3072/3073/3074) and evaluated with seeds 42/43/44 using 100 episodes per evaluation seed; a *task–run cell* is one task and one training run. The training run is the unit of replication throughout, the three evaluation seeds only measure within-run variability, and reported across-run means and dispersions are descriptive rather than population inference. The PLDM sweep trains one run for each of its 36 task–augmentation settings, with the

same evaluation seeds and episode count.

Diagnostic measurements. ATR uses 100 logged anchors, five perturbation draws, eight predicted steps, uniform horizon weights, within-anchor draw averaging, and a q90 summary across anchors. SMPR uses the same-state q90 tube, a global q35 radius among standardized endpoint states for selecting coordinate-proxy pairs, and margin 0.10. Appendix B gives the complete normalization and threshold protocol. The raw ATR defines the tube in SMPR; checkpoint thresholding uses the task-relative ATR in Equation (14).

Threshold evaluation. A Gaussian-sweep checkpoint meets the *success-rate criterion* when it attains at least 80% of the improvement from the unaugmented checkpoint to the best checkpoint at evaluation noise $\sigma = 0.08$, while losing no more than five percentage points on clean observations; both constants were fixed a priori. We select (t_R, t_M) on every nonempty proper subset of the four tasks and apply the thresholds unchanged to the remaining tasks, giving $4 + 6 + 4 = 14$ directional source/evaluation partitions from one, two, or three source tasks. Metrics first average training runs within each evaluation task and then weight tasks equally; checkpoint rows are never treated as independent replicates. Success rates are used to select and evaluate thresholds, but they are not inputs to ACPC, ATR, or SMPR.

For blur and resize, each task uses the Gaussian thresholds selected on the other three tasks. We compare 24 checkpoint pairs (four tasks $\times$ three runs $\times$ two shifts), each pairing the unaugmented checkpoint with the checkpoint trained at $\sigma_{\max} = 0.08$. A pair is labeled positive when success rate under the shift improves by at least five percentage points while clean success rate decreases by no more than five points. No blur- or resize-specific adjustment is made, and this analysis does not benchmark alternative signals such as encoder shift or one-step ACPC.

4.2 Planning performance under observation noise

At evaluation noise $\sigma = 0.08$, the unaugmented LeWM checkpoints lose 25.9 ± 2.4, 74.6 ± 5.6, 41.0 ± 1.4, and 22.1 ± 2.8 percentage points in success rate on TwoRoom, PushT, Reacher, and Cube, respectively. Gaussian augmentation recovers performance over a range of training levels whose location differs by task. Figure 1 compares success rate with ATR and SMPR across the complete sweep.

Every augmentation level that meets the success-rate criterion has lower ATR and higher SMPR than the corresponding unaugmented checkpoint, although neither diagnostic needs to change monotonically with augmentation strength. **Takeaway:** both diagnostics move with recovered planning performance, which motivates the combined score; whether ACPC actually uses action information and tracks planning-relevant quantities is established by the next two experiments (Sections 4.3 and 4.4).

4.3 Predicting the perturbation-induced error drift

Proposition 1 bounds, for each history pair, the perturbation-induced change in prediction error: the two H-step errors against the same logged future can differ by at most ACPC_H. This section tests whether measured ACPC is informative about that bounded change, the error drift $d = |e_{\tilde{h}} - e_h|$. We evaluate at the protocol horizon $H = 8$, the longest horizon with a complete logged common future for every anchor; Table 4 shows that the checkpoint-level conclusions are stable for $H \in \{1, 2, 4, 8\}$.

Data and protocol. The analysis uses the unaugmented checkpoint of each task and training run. Trajectories are partitioned into 16 disjoint groups. Each history pair contributes rows at Gaussian probe severities $\{0.02, 0.05, 0.08\}$ with two perturbation draws; zero-severity probes serve only as identity checks. A ridge model is fitted on 15 groups and evaluated on the held-out group, rotating through all 16, and all rows from one trajectory group stay together; we report the resulting out-of-group MAE.

Three nested regressions. Every model contains the probe severity and the encoder-history distance $\|z - \tilde{z}\|_2$.

- **Base** adds one-step ACPC under the recorded action: the information available without a multi-step rollout.
- **Base+Control$_8$** adds the strongest *destroyed-action control*: eight-step ACPC recomputed with *zeroed* actions (every action set to zero), *swapped* actions (the recorded actions of a different trajectory), or *shuffled* actions (the anchor's own actions in permuted temporal order). Each control performs the same eight-step rollout, so it carries rollout length without the recorded action sequence. "Strongest" is an oracle choice: in

each task–run cell we report whichever of the three controls attains the lowest test MAE, which makes the comparison conservative.

- **Base+ACPC$_8$** instead adds eight-step ACPC under the recorded actions—the only feature whose action sequence is the one that generated the logged future, and hence the feature matching the right-hand side of Proposition 1.

All eight-step features use uniform weights $\alpha_k = 1/8$ in Equation (3). If Base+ACPC$_8$ improves on Base, multi-step information helps; if it also improves on Base+Control$_8$, the gain is attributable to the recorded actions rather than to merely rolling out eight steps.

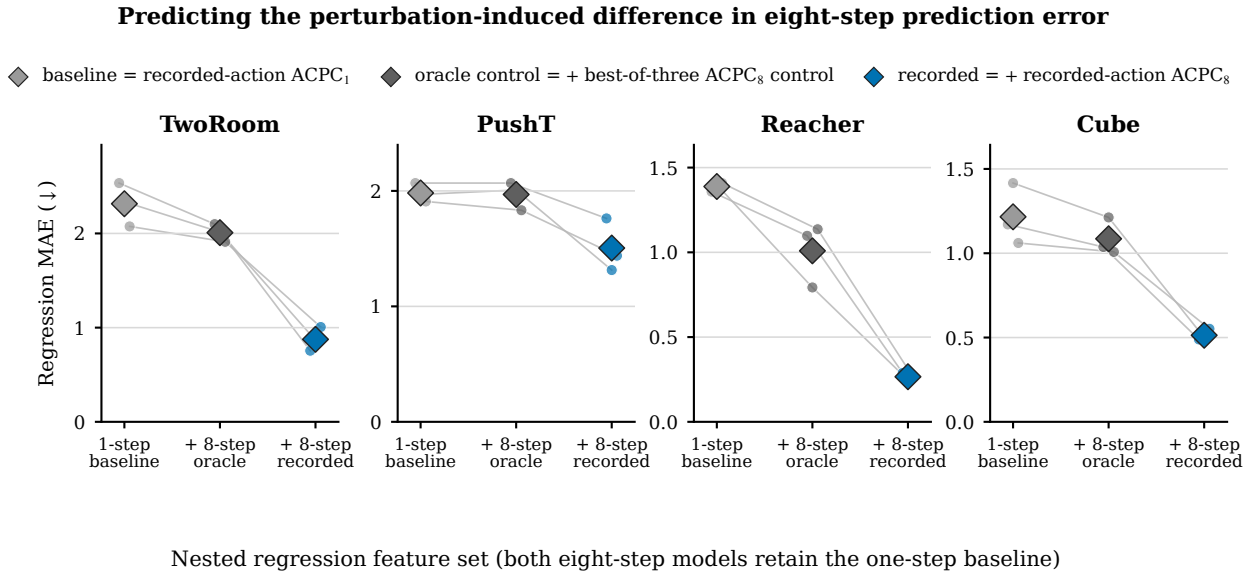

Figure 2: Out-of-group MAE for predicting the error drift d on the unaugmented checkpoints; lower is better. Small points are training runs, thin lines join the same run, and diamonds are task means. The in-figure legend labels correspond to Base, Base+Control$_8$, and Base+ACPC$_8$ as defined in Section 4.3. Base+ACPC$_8$ is lowest in all 12 task–run cells; latent-error scales are task-specific, so compare within a panel.

Results. Base+ACPC$_8$ attains the lowest out-of-group MAE in all 12 task–run cells (Figure 2). For each training run we average the four task-level relative MAE reductions equally and report the mean ± sample standard deviation of these *run-level equal-task summaries*: the reduction is $55.9 \pm 4.7\%$ relative to Base and $51.3 \pm 3.5\%$ relative to Base+Control$_8$. These results concern prediction of the error drift; they do not compare the absolute forecast accuracy of one-step and eight-step world models. Appendix C reports the task-level values and selected controls. **Takeaway:** eight-step ACPC carries information about the bounded error change that neither encoder shift, one-step ACPC, nor any same-horizon destroyed-action control provides.

4.4 ACPC and CEM decisions

This section tests whether ACPC is informative about the CEM quantities in Proposition 2. The audit compares the unaugmented checkpoint and the checkpoint trained at $\sigma_{\max} = 0.08$ on identical candidate pools. The reduced-budget configuration uses 100 histories per task and training run with $K = 64$ candidates, top-eight elites, and eight CEM steps; the full-budget configuration reuses a fixed set of 16 histories per task, shared across training conditions and runs, with the deployed $K = 300$, top-30, 30-step setting. *Candidate-conditioned* ACPC is computed along each candidate action sequence from the planner pool rather than the logged actions, with $H = 5$ to match the plan horizon (Section 4.1).

Soundness audit. Across the 9,600 reduced-budget records (100 histories × four tasks × three runs × two training conditions × four severities), the candidate-cost bound has no violations, and the winner and elite-set certificates never assert stability when the corresponding set changes. Identity probes—severity-zero duplicates of the clean history—give exactly zero five-step ACPC and zero first-action RMS. Whenever the elite-set certificate

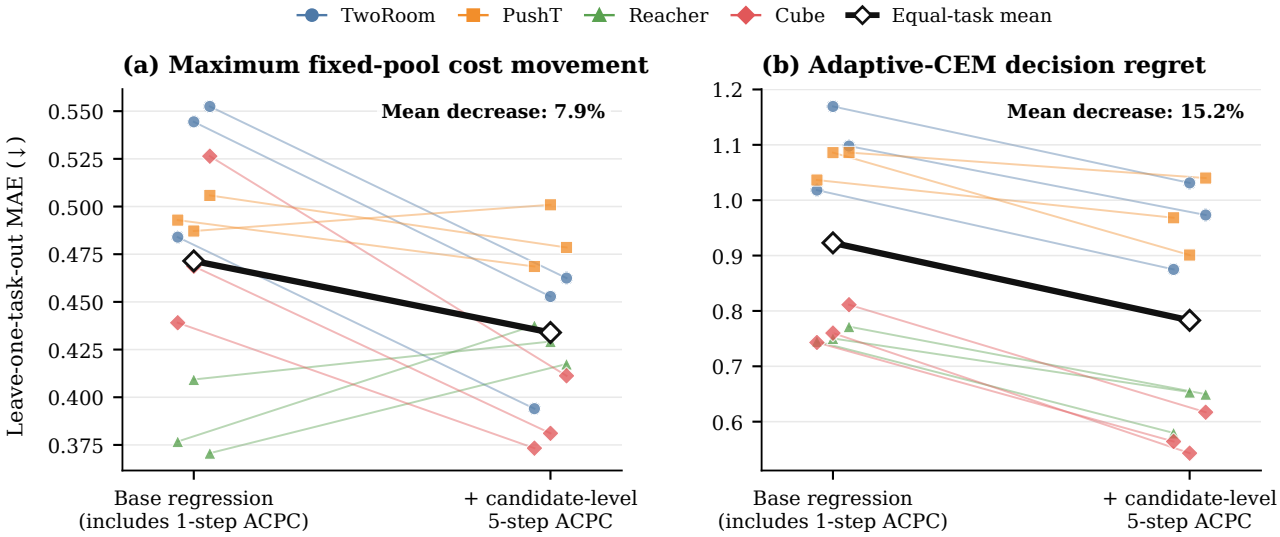

Figure 3: Leave-one-task-out prediction errors for two planner-facing targets, without and with candidate-conditioned five-step ACPC; lower is better. Thin lines connect the two errors of one task–run evaluation; the heavy black pair is the across-run mean of run-level equal-task MAEs, and the annotation is the mean relative decrease from Table 2. Errors are on the fitted $\log(1 + \text{target})$ scale; the first-action target is omitted because its gain is negligible.

remains valid, the nominal and perturbed CEM updates also remain aligned, as Corollary 1 requires; details are in Appendix G.

Incremental predictive value. We then test whether candidate-conditioned five-step ACPC adds information beyond a baseline containing perturbation severity, training condition, candidate-conditioned one-step ACPC, and the nominal top-1 margin (the clean cost gap between the best and second-best candidates). Within each training run, ridge models are fitted on three tasks and evaluated on the fourth, rotating over all tasks. The prediction targets are (i) the maximum candidate-cost movement in the fixed pool, (ii) the clean-history decision regret of adaptive CEM, and (iii) the RMS change of the first action of the final plan—the action MPC would execute. Table 2 reports the reduction in leave-one-task-out MAE, and Figure 3 shows the paired errors.

Table 2: Leave-one-task-out MAE decrease from adding candidate-conditioned five-step ACPC to the baseline regression of Section 4.4 (reduced-budget CEM audit, 9,600 records). Entries are the mean ± sample SD of run-level equal-task decreases on the fitted $\log(1 + \text{target})$ scale; the last column counts positive task–run cells out of 12. Higher is better.

Prediction target	Leave-one-task-out MAE decrease (%, mean ± sample SD, ↑)	Positive task–run cases (/12)
Maximum fixed-pool cost movement	7.9 ± 1.4	8/12
Adaptive-CEM decision regret	15.2 ± 2.0	12/12
Adaptive-CEM first-action change (RMS)	1.1 ± 0.5	10/12

Adding five-step ACPC reduces leave-one-task-out MAE for maximum fixed-pool cost movement by $7.9 \pm 1.4\%$, positive in 8 of 12 task–run cells and at the task-level mean on three of the four tasks. For adaptive CEM, it reduces MAE for positive decision regret by $15.2 \pm 2.0\%$, with an improvement in every task–run cell. The $1.1 \pm 0.5\%$ reduction for first-action change is too small to support an incremental claim. In all three analyses, the Spearman rank correlation with the target is higher for five-step than for one-step ACPC in every task–run cell.

Effect of augmentation on decisions. At perturbation severity 0.08, the task-level mean rate of retaining the best fixed-pool candidate is 0.92–0.96 after Gaussian-augmented training, compared with 0.02–0.13 for the unaugmented checkpoints. Mean positive decision regret is also lower with augmentation in every task–run cell: 3.64 ± 0.68 versus 227.04 ± 16.92 at reduced budget and 2.65 ± 1.43 versus 244.89 ± 27.08 at full budget (run-level equal-task summaries). Regret is measured in the model's squared latent goal cost, not simulator return, and latent-cost scales differ by task, so these contrasts are descriptive; Appendix G gives the task-level

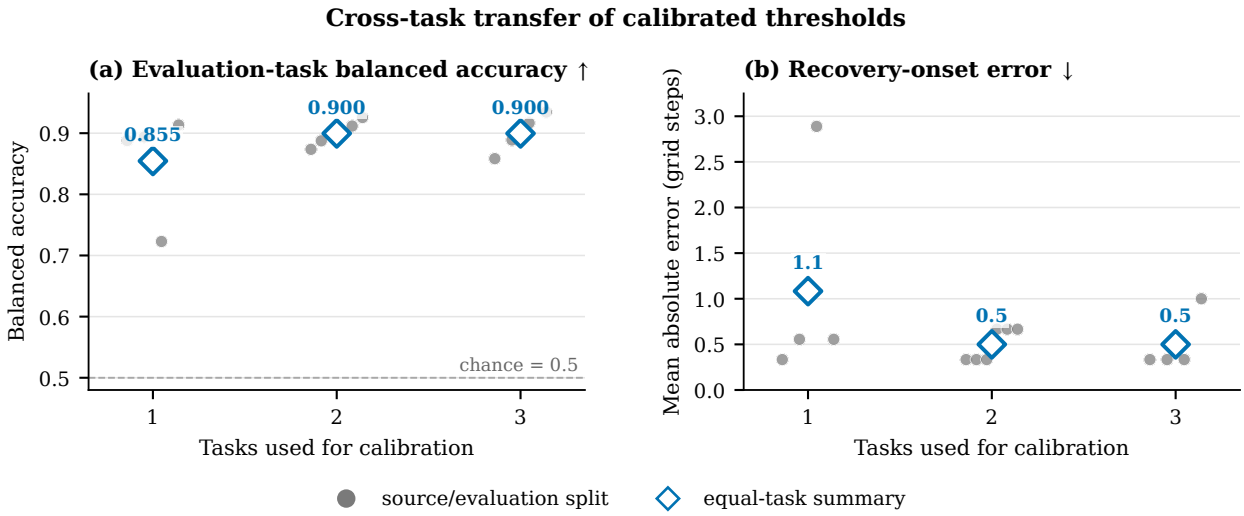

Figure 4: Cross-task transfer of ATR–SMPR thresholds over all 14 source/evaluation partitions; a checkpoint passes when relative ATR $\leq t_R$ and SMPR $\geq t_M$. (a) Balanced accuracy on checkpoints from evaluation tasks (chance is 0.5). (b) Absolute onset error in Gaussian-augmentation grid steps (one step is 0.01 in $\sigma_{\max}$). Gray points are individual partitions; diamonds are equal-task summaries per number of source tasks.

values. **Takeaway:** the deterministic certificates hold exactly, and candidate-conditioned ACPC adds cross-task predictive information about which decisions a visual perturbation will move, most clearly for decision regret.

4.5 Cross-task threshold transfer

This experiment tests whether ATR–SMPR thresholds selected on some tasks (*source tasks*) screen checkpoints from tasks excluded from calibration (*evaluation tasks*). Thresholds are selected on every nonempty proper subset of the four tasks and applied unchanged to the remaining tasks, giving the 14 directional partitions of Section 4.1 (Figure 4). A checkpoint counts as positive when it meets the success-rate criterion, and the *onset error* is the gap, in augmentation grid steps, between the first level accepted by the diagnostic and the first level meeting the criterion.

Balanced accuracy is 0.855, 0.900, and 0.900 when thresholds are selected from one, two, and three source tasks, respectively, with precision/recall of 0.910/0.854, 0.913/0.953, and 0.913/0.953. Mean absolute onset errors are 1.1, 0.5, and 0.5 grid steps; the largest individual partition–task–run errors are 5, 1, and 1 steps. With one source task, balanced accuracy ranges from 0.723 to 0.913 across the four possible sources—the weakest single source is Reacher (Table 9)—so calibration depends on task composition, while two and three source tasks perform similarly on average in this four-task study. **Takeaway:** the threshold-selection *procedure* transfers across these tasks; a single universal threshold value does not. Appendix D reports every partition.

4.6 Application to PLDM

We apply the same paired-history construction, eight-step rollout, ATR normalization, SMPR guard, success-rate criterion, and threshold-selection procedure to the complete PLDM Gaussian sweep (one training run per setting, so run-to-run variability is not estimated here). The analysis uses PLDM's encoder and predictor and does not depend on LeWM layers or training losses. For each PLDM evaluation task, thresholds are selected on the other three PLDM tasks and then fixed for its nine checkpoints.

Pooling all 36 held-out checkpoint decisions (four evaluation tasks × nine checkpoints) gives 0.836 balanced accuracy, 0.789 precision, and 0.882 recall. Task-level balanced accuracies are 0.938, 0.750, 0.500, and 0.875 for TwoRoom, PushT, Reacher, and Cube: on Reacher the transferred thresholds perform at chance. After normalizing ATR within each PLDM task, applying the LeWM thresholds selected on the corresponding three source tasks gives the same aggregate values (0.836/0.789/0.882). **Takeaway:** the equations and evaluation procedure carry over to a second joint-embedding architecture, but numerical calibration remains model-family specific, and the aggregate agreement above does not make thresholds architecture-invariant.

Table 3: The same ACPC analysis applied to the complete PLDM sweep (four tasks × nine augmentation levels, one training run each; 36 held-out decisions). Row 1 selects thresholds on the other three PLDM tasks; row 2 applies the corresponding LeWM thresholds after within-task ATR normalization. "False pass"/"false miss" count criterion-negative checkpoints accepted and criterion-positive checkpoints rejected.

Calibration	BA	Precision	Recall	False pass	False miss
PLDM thresholds, other three tasks	0.836	0.789	0.882	4	2
LeWM thresholds, PLDM-normalized	0.836	0.789	0.882	4	2

4.7 Transfer to blur and resize

The last experiment tests whether a Gaussian-calibrated change in the selective score orders checkpoint pairs in the same direction as planning success under visual shifts never used for calibration: blur ($k = 15$) and resize (scale 0.25), each at one fixed severity. For each task, thresholds selected on the other three Gaussian-noise tasks are applied to the 24 checkpoint pairs of Section 4.1; no blur or resize result is used to select or adjust any threshold.

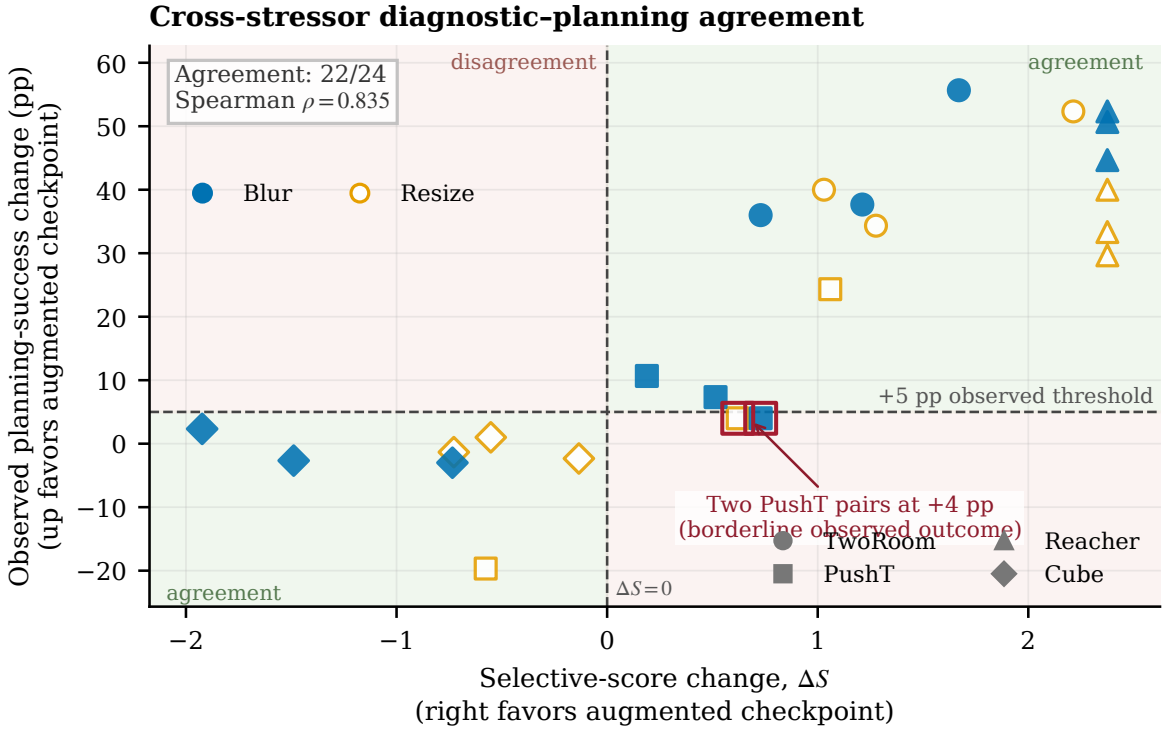

Figure 5: Gaussian-calibrated selective-score change ΔS versus the observed change in planning success under blur and resize. Each of the 24 points pairs an unaugmented checkpoint with its $\sigma_{\max} = 0.08$ counterpart; marker shape denotes task, color/fill denotes the shift. The diagnostic predicts improvement when $\Delta S > 0$; the observed positive label additionally requires no more than a five-point clean loss, which these axes cannot show. Background regions mark agreement.

The sign of ΔS agrees with the observed label on 22 of 24 pairs (0.889 balanced accuracy, 0.882 precision, 1.000 recall; balanced accuracy is 0.875 for blur and 0.900 for resize alone), and the continuous score change has Spearman's $\rho = 0.835$ with the change in success rate. The two discordant cases are both PushT pairs whose success rate improves by exactly four percentage points—one point below the fixed five-point label—while the selective score improves (Appendix E). **Takeaway:** the Gaussian-calibrated score preserves relative checkpoint ordering under these two shifts at the tested severities; it is not an absolute robustness classifier.

5 Discussion and limitations

What the evidence shows. The prediction experiment (Section 4.3) shows that recorded-action eight-step ACPC is informative about the error drift bounded by Proposition 1, beyond encoder shift, one-step ACPC, and the strongest same-horizon destroyed-action control; because the controls share the eight-step horizon, the gain cannot be attributed to rollout length alone. The CEM experiment (Section 4.4) verifies the deterministic cost bound exactly and shows that candidate-conditioned ACPC adds cross-task predictive information about decision regret, with a smaller gain for maximum cost movement and none for the first action. The transfer experiments (Sections 4.5 to 4.7) show that the threshold-selection procedure—not any single threshold value—transfers across tasks, to blur and resize, and to PLDM after per-family recalibration.

Scope and limitations. All planner results are model-cost and candidate-selection statements under paired evaluations, not environment success rates, and the adaptive-CEM guarantee is conditional on pool alignment up to the first failed elite certificate. SMPR guards only the declared coordinate proxies, so stable but incorrect predictions remain possible. Three training runs per task (and a single run per PLDM setting) support consistency checks rather than precise variability estimates; the full-budget CEM analysis reuses 16 histories per task; and no matched runs without diagnostic computation were collected, so incremental wall-clock overhead is not estimated. Blur and resize are each tested at one severity, and the one-source calibration range in Section 4.5 shows that source-task composition matters.

Future work. Important next steps are direct task outcomes for ACPC-informed planner interventions, multiple severities per visual shift, richer different-state proxies, and additional model families with different objectives and planner interfaces.

6 Conclusion

ACPC measures how a same-state visual perturbation changes a world model's predicted trajectory under fixed actions, exact inequalities relate that change to prediction error and to squared latent goal cost, and a state-coordinate margin rejects constant-representation collapse on the tested proxies. Across four tasks, recorded-action ACPC predicts perturbation-induced error changes beyond one-step and destroyed-action controls, tracks CEM cost and decision changes, and supports thresholds that transfer to held-out tasks, to blur and resize, and—after model-family recalibration—to PLDM. We see ACPC as a low-cost audit step before a checkpoint is deployed behind a planner: it flags visual predictive sensitivity that encoder-level checks cannot see, using only logged data and the frozen model.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zholus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[3] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.

[4] Jiaheng Chen. ATM: Action-consistency transfer matrix for diagnosing and improving latent world models, 2026.

[5] Yuxin Chen, Jianglan Wei, Chenfeng Xu, Boyi Li, Masayoshi Tomizuka, Andrea Bajcsy, and Ran Tian. Reimagination with test-time observation interventions: Distractor-robust world model predictions for visual model predictive control, 2025.

[6] Tom Dupuis, Jaonary Rabarisoa, Quoc-Cuong Pham, and David Filliat. VIBR: Learning view-invariant value functions for robust visual control. In *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 658–682. PMLR, 2023.

[7] T. W. Epps and Lawrence B. Pulley. A test for normality based on the empirical characteristic function. *Biometrika*, 70(3):723–726, 1983.

[8] Carles Gelada, Saurabh Kumar, Jacob Buckman, Ofir Nachum, and Marc G. Bellemare. DeepMDP: Learning continuous latent space models for representation learning. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2170–2179. PMLR, 2019.

[9] Hafez Ghaemi, Eilif Benjamin Muller, and Shahab Bakhtiari. seq-JEPA: Autoregressive predictive learning of invariant-equivariant world models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[10] Christopher Grimm, Andre Barreto, Satinder Singh, and David Silver. The value equivalence principle for model-based reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, 2020.

[11] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640(8059):647–653, 2025.

[12] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.

[13] Nicklas Hansen and Xiaolong Wang. Generalization in reinforcement learning by soft data augmentation. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 13611–13617, 2021.

[14] David Klindt, Yann LeCun, and Randall Balestriero. When does LeJEPA learn a world model?, 2026.

[15] Ilya Kostrikov, Denis Yarats, and Rob Fergus. Image augmentation is all you need: Regularizing deep reinforcement learning from pixels. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[16] Yann LeCun. A path towards autonomous machine intelligence. OpenReview preprint, 2022.

[17] Etai Littwin, Omid Saremi, Madhu Advani, Vimal Thilak, Preetum Nakkiran, Chen Huang, and Joshua Susskind. How JEPA avoids noisy features: The implicit bias of deep linear self distillation networks, 2024.

[18] Lucas Maes, Quentin Le Lidec, Luiz Facury, Nassim Massaudi, Ayush Chaurasia, Francesco Capuano, Richard Gao, Taj Gillin, Dan Haramati, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel: A platform for reproducible world modeling research and evaluation, 2026.

[19] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.

[20] Mingyu Park, Samyeul Noh, Hyun Myung, and Donghwan Lee. Zero-shot visual generalization in model-based reinforcement learning via latent consistency. OpenReview (ICLR 2026 submission), 2025.

[21] Bo-Kai Ruan, Teng-Fang Hsiao, Ling Lo, and Hong-Han Shuai. Is the future compatible? diagnosing dynamic consistency in world action models, 2026.

[22] Finn Rasmus Schäfer, Korbinian Moller, Yuan Gao, Christian Oefinger, Sebastian Schmidt, and Johannes Betz. Imagined rollouts are kinematic, not dynamic: A diagnosis of long-horizon world-model failure, 2026. RSS Robot World Model Workshop 2026.

[23] Gawon Seo, Dongwon Kim, and Suha Kwak. ACID: Action consistency via inverse dynamics for planning with world models, 2026.

[24] Yutaka Shimizu and Masayoshi Tomizuka. Bisimulation metric for model predictive control. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[25] Vlad Sobal, Jyothir S V, Siddhartha Jalagam, Nicolas Carion, Kyunghyun Cho, and Yann LeCun. Joint embedding predictive architectures focus on slow features, 2022.

[26] Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Stress-testing offline reward-free reinforcement learning: A case for planning with latent dynamics models. In *WRL@ICLR 2025*, 2025.

[27] Leonardo F. Toso, Davit Shadunts, Yunyang Lu, Nihal Sharma, Donglin Zhan, Nam H. Nguyen, and James Anderson. Learning invariant visual representations for planning with joint-embedding predictive world models. *arXiv preprint arXiv:2602.18639*, 2026.

[28] Hugues Van Assel, Mark Ibrahim, Tommaso Biancalani, Aviv Regev, and Randall Balestriero. Joint embedding vs reconstruction: Provable benefits of latent space prediction for self-supervised learning, 2025.

[29] Claas A. Voelcker, Anastasiia Pedan, Arash Ahmadian, Romina Abachi, Igor Gilitschenski, and Amir-Massoud Farahmand. Calibrated value-aware model learning with probabilistic environment models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 61745–61768. PMLR, 2025.

[30] Zijie Wang, Wei Zhang, Weiming Zhang, Fanqi Zhang, Xiao Tan, Yipeng Qin, and Guanbin Li. World models as group actions, 2026.

[31] Han Yan, Zishang Xiang, Zeyu Zhang, and Hao Tang. MWM: Mobile world models for action-conditioned consistent prediction, 2026.

[32] Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[33] Amy Zhang, Rowan T. McAllister, Roberto Calandra, Yarin Gal, and Sergey Levine. Learning invariant representations for reinforcement learning without reconstruction. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[34] Zhenghao Zhang, Yuanxiang Wang, Zhenyu Guan, Yujia Yang, Bingkang Shi, Tianyu Zong, Hongzhu Yi, Guoqing Chao, Xingchen Chen, Tiankun Yang, Chenxi Bao, Tao Yu, Jingjing Zhou, and Jungang Xu. Delta-JEPA: Learning action-sensitive world models via latent difference decoding, 2026.

A Candidate-Cost and Fixed-Pool Analysis

Proof of Proposition 2. For one candidate, suppress the index and factor the squared-distance difference:

$$|\tilde{C} - C| = \big|\|\tilde{x} - g\|_2^2 - \|x - g\|_2^2\big| \tag{17}$$
$$= \big|\langle \tilde{x} - x, \tilde{x} + x - 2g\rangle\big| \tag{18}$$
$$\le \|\tilde{x} - x\|_2 \, \|\tilde{x} + x - 2g\|_2 \tag{19}$$
$$\le r\big(\|x - g\|_2 + \|\tilde{x} - g\|_2\big) = b. \tag{20}$$

The first inequality is Cauchy–Schwarz and the second is the triangle inequality. Thus, for nominal winner w and competitor j,

$$\tilde{C}_j - \tilde{C}_w \ge C_j - C_w - |\tilde{C}_j - C_j| - |\tilde{C}_w - C_w| \ge C_j - C_w - b_j - b_w.$$

For the perturbed winner $\tilde{w}$,

$$C_{\tilde{w}} \le \tilde{C}_{\tilde{w}} + b_{\tilde{w}} \le \tilde{C}_w + b_{\tilde{w}} \le C_w + b_w + b_{\tilde{w}},$$

which proves Equation (6); nonnegativity follows from the optimality of w on the nominal pool. Positivity of Equation (7) makes every perturbed competitor strictly more costly than w. The same argument for every $i \in \mathcal{E}$ and $j \notin \mathcal{E}$ proves that Equation (8) preserves the exact elite set. Strict inequalities avoid relying on implementation-specific tie breaking.

Proof of Corollary 1. At iteration zero, the proposals are identical by assumption. Common random numbers therefore produce the same ordered action pool. If the elite certificate holds, both branches select the same action vectors as elites. The CEM mean/variance update is a deterministic function of those vectors, so the next proposals are identical. Repeating this argument establishes pool and proposal alignment by induction. If a certificate fails, equality of the elite sets is unknown; the induction stops and no later shared-pool statement is made.

Relation to the weighted rollout radius. The planner bound uses the displacement at the cost readout horizon. For the weighted stack in Equation (3), $\sqrt{\alpha_H}\, r_j \le \mathrm{ACPC}_H$ whenever $\alpha_H > 0$. The planner experiments record r_j directly, avoiding the looser $1/\sqrt{\alpha_H}$ conversion. This does not require a future target, but it does require evaluating the candidate under both matched histories.

Sharpness without additional assumptions. Neither principal bound admits a uniformly smaller right-hand side from the available quantities alone. For a unit vector u, common target $Y = 0$, and two stacked predictions au and bu with $a, b \ge 0$, the reverse-triangle bound in Equation (4) holds with equality. Likewise, setting $g = 0$, $x = au$, and $\tilde{x} = bu$ makes $|\tilde{C} - C| = |b - a|(a + b) = r(\|x - g\|_2 + \|\tilde{x} - g\|_2)$.

B Evaluation and Diagnostic Protocol

These details define the fixed evaluation and diagnostic protocol used by the main tables.

Evaluation grid. LeWM is evaluated on PushT, TwoRoom, Reacher, and Cube with evaluation seeds 42/43/44 and 100 episodes per seed. The main evaluation perturbs observation pixels with Gaussian noise at standard deviation 0.08 while keeping the goal image clean. Checkpoints trained with noise use full-sequence input-side Gaussian augmentation with $\sigma_{\max} \in \{0.01, \ldots, 0.08\}$.

PLDM instantiation. PLDM uses the same four-task, nine-checkpoint Gaussian grid, evaluation seeds, trajectory count, paired-history construction, $H = 8$ rollout statistic, and SMPR definition. For its cross-task analysis, ATR is divided by its value for the corresponding unaugmented PLDM checkpoint, and thresholds are selected on the other three PLDM tasks. The evaluation task's success rates never enter threshold selection. A second test applies the corresponding LeWM threshold pair after this within-task PLDM normalization. Raw thresholds are not assumed to be shared across model families.

Success-rate criterion. Within each task and training run, let B and M be the success rates at evaluation noise $\sigma = 0.08$ for the unaugmented checkpoint and the best checkpoint in the sweep. A checkpoint meets the success-rate criterion when its noisy-observation success rate is at least $B + 0.8(M - B)$ and its clean success rate is no more than five percentage points below that of the unaugmented checkpoint. The green spans in Figure 1 require this criterion for a majority of runs.

ATR protocol. ATR is computed for each task, training run, and checkpoint from one weighted-stacked horizon radius per anchor. The default is $H = 8$ with uniform $\alpha_k = 1/H$. Each radius is divided by that anchor's q50 clean observed-transition scale, including the transition from the last history observation to the first future observation; multiple noise draws are first averaged within the anchor, and q90 is then taken across anchors. This is the raw ATR in Equation (11). It defines the same-state tube used by SMPR. For the sweep display and within-family cross-task calibration, raw ATR is divided by the corresponding unaugmented value as in Equation (14) before thresholds or run summaries are computed. The numerical stabilizers are $\varepsilon_{\mathrm{norm}} = 10^{-8}$ in Equation (10) and $\varepsilon_{\mathrm{score}} = 10^{-12}$ in Equation (15). Table 4 reports how the checkpoint-level ATR reduction depends on the rollout horizon and the summary quantile.

Table 4: Horizon and quantile sensitivity of the ATR reduction at fixed evaluation noise $\sigma = 0.08$. Each entry is the median, over the three training runs, of the ratio between the ATR of the checkpoint trained at the highest augmentation level ($\sigma_{\max} = 0.08$) and that of the unaugmented checkpoint; lower means a larger reduction. These values are descriptive and do not retune any threshold.

	$q = 0.90$ by horizon				$H = 8$ by quantile		
Task	H1	H2	H4	H8	q80	q90	q95
TwoRoom	0.05	0.04	0.05	0.06	0.06	0.06	0.06
PushT	0.06	0.07	0.06	0.09	0.07	0.09	0.09
Reacher	0.02	0.03	0.03	0.02	0.02	0.02	0.02
Cube	0.04	0.03	0.04	0.04	0.03	0.04	0.04

Table 6: Summary of the Gaussian-augmentation sweep (nine checkpoints per task: no augmentation plus eight noise levels; Figure 1 shows every level). "Best" is the level with the highest mean planning success at evaluation noise $\sigma = 0.08$; arrows give the change from the unaugmented checkpoint to that level. Relative ATR is lower-is-better, SMPR higher-is-better; the last column lists levels meeting the success-rate criterion (Section 4.1).

Task	No-aug. success (%)	Best success (%)	Best $\sigma_{\max}$	Relative ATR	SMPR	Levels meeting criterion
TwoRoom	68.8	97.1	0.08	1.00→0.07	0.11→0.98	0.01–0.08
PushT	7.2	86.8	0.06	1.00→0.11	0.28→1.00	0.03–0.08
Reacher	18.2	83.3	0.07	1.00→0.03	0.37→0.99	0.03–0.08
Cube	43.1	66.0	0.03	1.00→0.26	0.07→0.98	0.03–0.07

SMPR protocol. SMPR follows Equation (13). For each eligible anchor, the pair is its nearest proxy-label-crossing neighbor whose standardized state distance is no greater than the global q35 radius. The two clean histories are rolled out under the anchor's recorded action sequence, their distance is normalized by the anchor's clean transition scale, and the pair passes only when that distance is strictly greater than the raw q90 same-state tube plus 0.10.

Table 5 makes the pairing rule explicit. The logged state is taken at the end of the observed eight-step future and standardized coordinate-wise by the dataset preprocessor. We compute all off-diagonal Euclidean distances in that task's full standardized state vector and use their global q35 as the neighborhood radius. Proxy labels use median splits computed within the fixed 100-endpoint anchor batch (anchor seed 9101); each label therefore describes the endpoint reached by that trajectory's own recorded actions. After neighbor selection, both starting histories are rolled out under the anchor's actions for the latent-distance test. An anchor is skipped only when no label-crossing state lies inside the neighborhood. The eligible counts are constant across the evaluated checkpoints and training runs because pairing is fixed by the logged states.

Table 5: Implemented state-coordinate proxies for SMPR. The state is taken at the eighth observed future step and standardized coordinate-wise with full-dataset statistics. Counts give eligible and skipped anchors in the fixed 100-anchor audit; these labels are not semantic or action-relevance annotations.

Task	HDF5 key (dim.)	Implemented proxy label $\ell(y)$	Eligible / skipped
TwoRoom	`pos_agent` (2)	Median split of the standardized agent x coordinate.	61 / 39
PushT	`state` (7)	Three median bits from standardized block x, block y, and block angle (slice `state[2:5]`).	98 / 2
Reacher	`observation` (6)	Three median bits from the two standardized joint-velocity coordinates and their Euclidean norm (slice `observation[4:6]`).	100 / 0
Cube	`observation` (28)	For standardized observation $\bar{o}$, let $r = \bar{o}_{0:3} - \bar{o}_{25:28}$; use median bits of r_1, r_2, and $\|r\|_2$.	100 / 0

Threshold grids. For cross-task evaluation, candidate thresholds are $t_R \in \{0.05, 0.075, 0.10, 0.15, 0.20, 0.30\}$ for task-relative ATR and $t_M \in \{0.80, 0.85, 0.90, 0.95\}$ for SMPR. Within each source-task subset, the selection criterion lexicographically minimizes mean absolute onset error, false-early and false-late counts, then maximizes balanced accuracy. The selected pair is applied without modification to every evaluation task. No evaluation-task label or blur/resize outcome enters threshold selection.

Reproducibility. All summary figures and tables are rebuilt deterministically by the scripts documented in `paper1/scripts/README.md`. Checkpoint-level diagnostics additionally require the released checkpoints and datasets. Machine-readable summaries record training seeds, evaluation seeds, and protocol hashes.

Table 7: Relative reduction in out-of-group MAE from Base+ACPC$_8$ when predicting the error drift d on the unaugmented checkpoints (protocol and model definitions in Section 4.3). Base+Control$_8$ uses the per-cell oracle control; higher is better. The last column counts task–run cells in which Base+ACPC$_8$ is best.

Training run (seed)	vs. Base	vs. Base+Control$_8$	Task–run cells improved
3072	55.5%	51.4%	4/4
3073	60.8%	54.8%	4/4
3074	51.4%	47.7%	4/4
mean ± SD	55.9 ± 4.7%	51.3 ± 3.5%	12/12

Table 8: Out-of-group MAE (16-fold leave-one-trajectory-group-out) for predicting the error drift d on the unaugmented checkpoints; lower is better, model definitions in Section 4.3. "Selected control" is the destroyed-action control chosen by the per-cell oracle; "paired wins" counts the folds (of 16) in which Base+ACPC$_8$ beats Base+Control$_8$.

Task	run (seed)	Base	Base +Control$_8$	Base +ACPC$_8$	selected control	paired wins
TwoRoom	3072	2.535	2.099	**0.755**	shuffled actions	15/16
TwoRoom	3073	2.336	2.015	**0.866**	shuffled actions	15/16
TwoRoom	3074	2.075	1.911	**1.006**	swapped actions	13/16
PushT	3072	2.068	2.068	**1.762**	shuffled actions	11/16
PushT	3073	1.969	2.008	**1.315**	zeroed actions	13/16
PushT	3074	1.909	1.833	**1.439**	zeroed actions	13/16
Reacher	3072	1.358	1.097	**0.288**	shuffled actions	16/16
Reacher	3073	1.399	0.793	**0.247**	shuffled actions	16/16
Reacher	3074	1.409	1.136	**0.263**	shuffled actions	16/16
Cube	3072	1.171	1.037	**0.489**	zeroed actions	14/16
Cube	3073	1.416	1.212	**0.500**	zeroed actions	14/16
Cube	3074	1.061	1.008	**0.552**	shuffled actions	14/16

C Prediction-Error Scope and Controls

The common-future inequality in Proposition 1 has zero numerical violations in any logged eight-step or candidate-conditioned five-step task×run×training-condition cell; this is an implementation invariant, not an independent statistical success count.

Logged-future prediction and candidate planning are different estimands: the former predicts the error drift against an independent action-matched observed future, whereas the latter compares candidates from the actual planner pool. We do not infer one result from the other; Section 4.4 supplies the planner-facing evidence with same-pool, candidate-conditioned features.

The prediction-error analysis is restricted to the unaugmented checkpoints and the listed visual probes. Every training run improves on all four tasks.

D Cross-Task Threshold Partitions

The 14 rows below are exhaustive: selecting one, two, or three of four tasks as sources creates $4 + 6 + 4$ directional partitions. Each evaluation task retains every training run and all nine checkpoints. The apparent sample size is therefore not inflated by treating checkpoint rows as independent observations.

E Cross-Stressor Pairs and Discordant Cases

Both discordant rows lie near the success-rate criterion: PushT run 3072 under resize and PushT run 3074 under blur each improve success rate by four percentage points, one point below the five-point criterion, while their selective scores increase. The complete table shows these cases rather than only the aggregate classification metrics.

Table 9: Cross-task evaluation of the selective ACPC rule over all 14 source/evaluation partitions. Thresholds (t_R, t_M) are selected on the source tasks and applied unchanged to all remaining tasks; metrics first average training runs within each evaluation task and then weight tasks equally. Onset error is in Gaussian-augmentation grid steps (one step is 0.01 in $\sigma_{\max}$).

Source tasks	Evaluation tasks	t_R	t_M	BA	P / R	Onset error (steps) mean / max
TwoRoom	PushT, Reacher, Cube	0.3	0.95	0.888	0.884 / 0.981	0.3 / 1.0
PushT	TwoRoom, Reacher, Cube	0.3	0.95	0.894	0.902 / 0.956	0.6 / 1.0
Reacher	TwoRoom, PushT, Cube	0.1	0.95	0.723	0.906 / 0.540	2.9 / 5.0
Cube	TwoRoom, PushT, Reacher	0.3	0.95	0.913	0.950 / 0.938	0.6 / 1.0
TwoRoom, PushT	Reacher, Cube	0.3	0.95	0.874	0.853 / 1.000	0.3 / 1.0
TwoRoom, Reacher	PushT, Cube	0.3	0.95	0.887	0.873 / 0.972	0.3 / 1.0
TwoRoom, Cube	PushT, Reacher	0.3	0.95	0.903	0.925 / 0.972	0.3 / 1.0
PushT, Reacher	TwoRoom, Cube	0.3	0.95	0.896	0.901 / 0.935	0.7 / 1.0
PushT, Cube	TwoRoom, Reacher	0.3	0.95	0.912	0.952 / 0.935	0.7 / 1.0
Reacher, Cube	TwoRoom, PushT	0.3	0.95	0.926	0.972 / 0.907	0.7 / 1.0
TwoRoom, PushT, Reacher	Cube	0.3	0.95	0.858	0.802 / 1.000	0.3 / 1.0
TwoRoom, PushT, Cube	Reacher	0.3	0.95	0.889	0.905 / 1.000	0.3 / 1.0
TwoRoom, Reacher, Cube	PushT	0.3	0.95	0.917	0.944 / 0.944	0.3 / 1.0
PushT, Reacher, Cube	TwoRoom	0.3	0.95	0.935	1.000 / 0.869	1.0 / 1.0

Table 10: Transfer of the Gaussian-calibrated selective score to blur and resize. For each task, thresholds are selected on the other three Gaussian-noise tasks and fixed; each pair compares the $\sigma_{\max} = 0.08$ checkpoint with its unaugmented counterpart. "Discordant" counts pairs whose score-change sign disagrees with the observed success-rate label (Section 4.7).

Visual shift	Pairs	BA	Precision / recall	Spearman	Discordant
All pairs	24	0.889	0.882 / 1.000	0.835	2
Blur	12	0.875	0.889 / 1.000	0.873	1
Resize	12	0.900	0.875 / 1.000	0.746	1

Table 11: All 24 LeWM checkpoint pairs evaluated under blur and resize. $\mathrm{ATR}_{\mathrm{rel}}$ and SMPR are measured for the checkpoint trained with Gaussian augmentation ($\sigma_{\max} = 0.08$); ΔP and ΔS are its success-rate and selective-score changes relative to the unaugmented checkpoint. A pair is positive when $\Delta P \geq 5$ percentage points with at most a five-point clean loss. Daggers mark the two discordant pairs.

Task	Seed	Shift	$\mathrm{ATR}_{\mathrm{rel}}$	SMPR	ΔP	ΔS	Outcome (success / score)
TwoRoom	3072	blur	0.499	0.885	55.7	1.670	positive / positive
TwoRoom	3072	resize	0.336	0.967	52.3	2.214	positive / positive
TwoRoom	3073	blur	0.781	0.262	36.0	0.729	positive / positive
TwoRoom	3073	resize	0.691	0.361	40.0	1.030	positive / positive
TwoRoom	3074	blur	0.636	0.541	37.7	1.212	positive / positive
TwoRoom	3074	resize	0.617	0.656	34.3	1.277	positive / positive
PushT	3072	blur	0.943	0.939	10.7	0.189	positive / positive
PushT	3072	resize	0.814	0.969	4.0	0.620	nonpositive / positive†
PushT	3073	blur	0.846	0.918	7.3	0.515	positive / positive
PushT	3073	resize	0.682	0.969	24.3	1.060	positive / positive
PushT	3074	blur	0.781	0.959	4.0	0.729	nonpositive / positive†
PushT	3074	resize	1.173	0.939	-19.7	-0.577	nonpositive / nonpositive
Reacher	3072	blur	0.155	0.990	50.7	2.375	positive / positive
Reacher	3072	resize	0.210	0.990	29.7	2.375	positive / positive
Reacher	3073	blur	0.176	0.990	52.3	2.375	positive / positive
Reacher	3073	resize	0.207	0.990	40.0	2.375	positive / positive
Reacher	3074	blur	0.190	0.990	44.7	2.375	positive / positive
Reacher	3074	resize	0.195	0.990	33.3	2.375	positive / positive
Cube	3072	blur	1.447	0.330	-2.7	-1.488	nonpositive / nonpositive
Cube	3072	resize	1.166	0.530	1.0	-0.552	nonpositive / nonpositive
Cube	3073	blur	1.577	0.260	2.3	-1.923	nonpositive / nonpositive
Cube	3073	resize	1.218	0.560	-1.3	-0.728	nonpositive / nonpositive
Cube	3074	blur	1.220	0.660	-3.0	-0.735	nonpositive / nonpositive
Cube	3074	resize	1.040	0.770	-2.3	-0.134	nonpositive / nonpositive

F Local Gaussian Sensitivity as a Mechanistic Interpretation

For a small isotropic observation perturbation $\delta x \sim \mathcal{N}(0, \sigma^2 I)$, first-order linearization of encoder E followed by rollout map G gives

$$\mathbb{E}\left[\|\Delta G\|_2^2\right] \approx \sigma^2 \|J_G J_E\|_F^2. \tag{21}$$

This explains one mechanism by which Gaussian noise training can contract ACPC: it reduces the composed local sensitivity of the encoder–predictor path. It is a local approximation, not a global robustness bound.

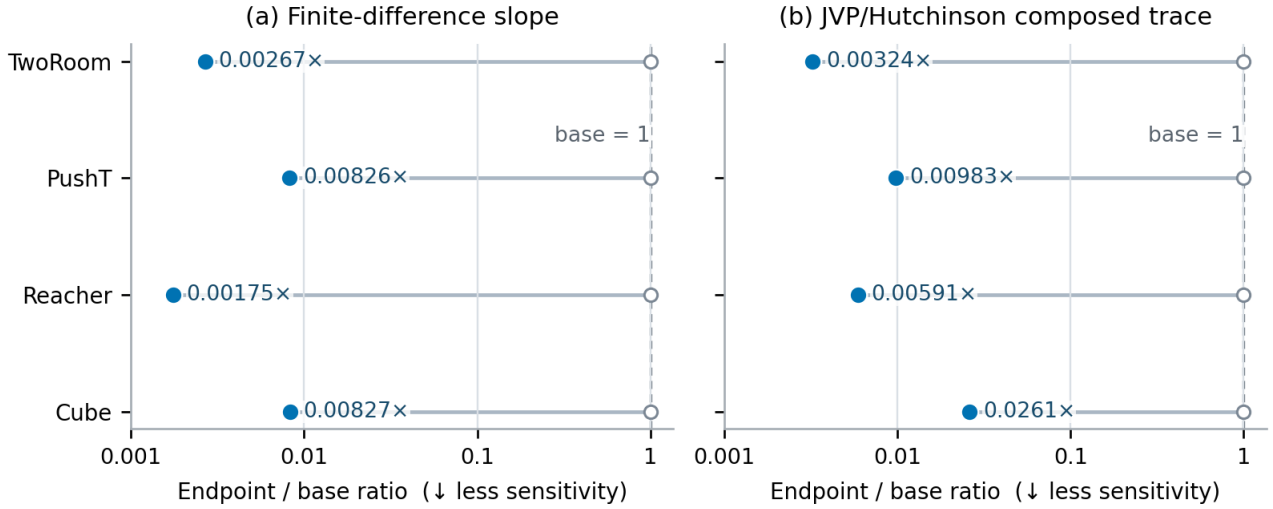

Figure 6: Local-sensitivity ratio of the checkpoint trained with Gaussian augmentation to the unaugmented checkpoint, estimated by finite differences and by a JVP/Hutchinson estimate of the composed Jacobian trace. Values below one indicate contraction after augmentation. The two estimators agree in direction for every task; these ratios do not establish task performance.

G Fixed-Pool Candidate Stability

The CEM analysis does not use task-success labels. It compares checkpoints unaugmented and $\sigma_{\max} = 0.08$ checkpoints on all four tasks. Fixed-pool shards capture the ordered initial proposal, evaluate the nominal and perturbed histories through one shared cost path, and record exact same-winner and elite-set events for 100 histories per task, run, and training condition at severities $0, 0.02, 0.05, 0.08$. Adaptive shards use a common initial proposal and common random numbers. Proposal alignment is asserted only while all preceding elite updates remain certified.

For nominal final plan $\mathbf{a}$ and perturbed plan $\tilde{\mathbf{a}}$, the reported positive clean-history regret is

$$\left[C_h(\tilde{\mathbf{a}}) - C_h(\mathbf{a})\right]_+.$$

It measures decision degradation in the model's latent goal cost, not an environment outcome. First-action RMS is reported separately. The full-budget evaluation keeps the same definitions but uses 16 fixed histories per task shared across training conditions and runs, $K = 300$, top-30 elites, and 30 CEM steps. Whole-shard runtimes and forward-call counts are retained for provenance, but without matched timing runs that omit the diagnostic they do not estimate incremental overhead.

Across 9,600 joined reduced-budget rows there are zero squared-distance-bound violations, zero false top-1 certificates, and zero false elite certificates. Identity probes have zero five-step displacement and zero first-action RMS; all identity updates remain aligned. These checks establish implementation soundness and are not additional independent statistical samples.

At severity 0.08, the sufficient top-1 certificate covers 10–70% of histories from checkpoints trained with augmentation across task–run blocks (and none from the unaugmented checkpoints); elite-set coverage is 0–30% with augmentation. Coverage is lower than realized stability because these conditions are sufficient but not necessary.

Table 12: CEM decision statistics at Gaussian perturbation severity 0.08. Values are mean ± sample SD across independent training runs after histories are averaged within each run. Reduced-budget rows use 100 histories, $K = 64$, top-8 elites, and eight CEM steps; full-budget rows use the same 16 histories per task with $K = 300$, top-30 elites, and 30 steps. Regret is measured with the model's squared latent goal cost and is not an environment success rate.

	Best candidate unchanged		Reduced-budget regret		Full-budget regret	
Task	No aug.	$\sigma_{\max}$=0.08	No aug.	$\sigma_{\max}$=0.08	No aug.	$\sigma_{\max}$=0.08
TwoRoom	0.13±0.03	0.93±0.02	203.81±9.97	8.47±2.75	224.97±30.56	1.75±1.44
PushT	0.13±0.09	0.92±0.01	167.50±29.53	4.30±0.28	220.14±32.97	7.91±4.23
Reacher	0.07±0.01	0.96±0.04	281.12±20.24	0.47±0.14	269.44±22.21	0.28±0.20
Cube	0.02±0.02	0.92±0.03	255.72±12.28	1.31±0.11	265.03±28.08	0.63±0.22
Equal-task mean	0.09±0.03	0.93±0.01	227.04±16.92	3.64±0.68	244.89±27.08	2.65±1.43

H Qualitative Neighborhood Visualization

The two-dimensional view complements the quantitative analysis by showing how repeated perturbations of the same anchors are arranged before and after noise training. Because t-SNE does not preserve metric geometry, all reported evidence remains in the original projected-rollout space.

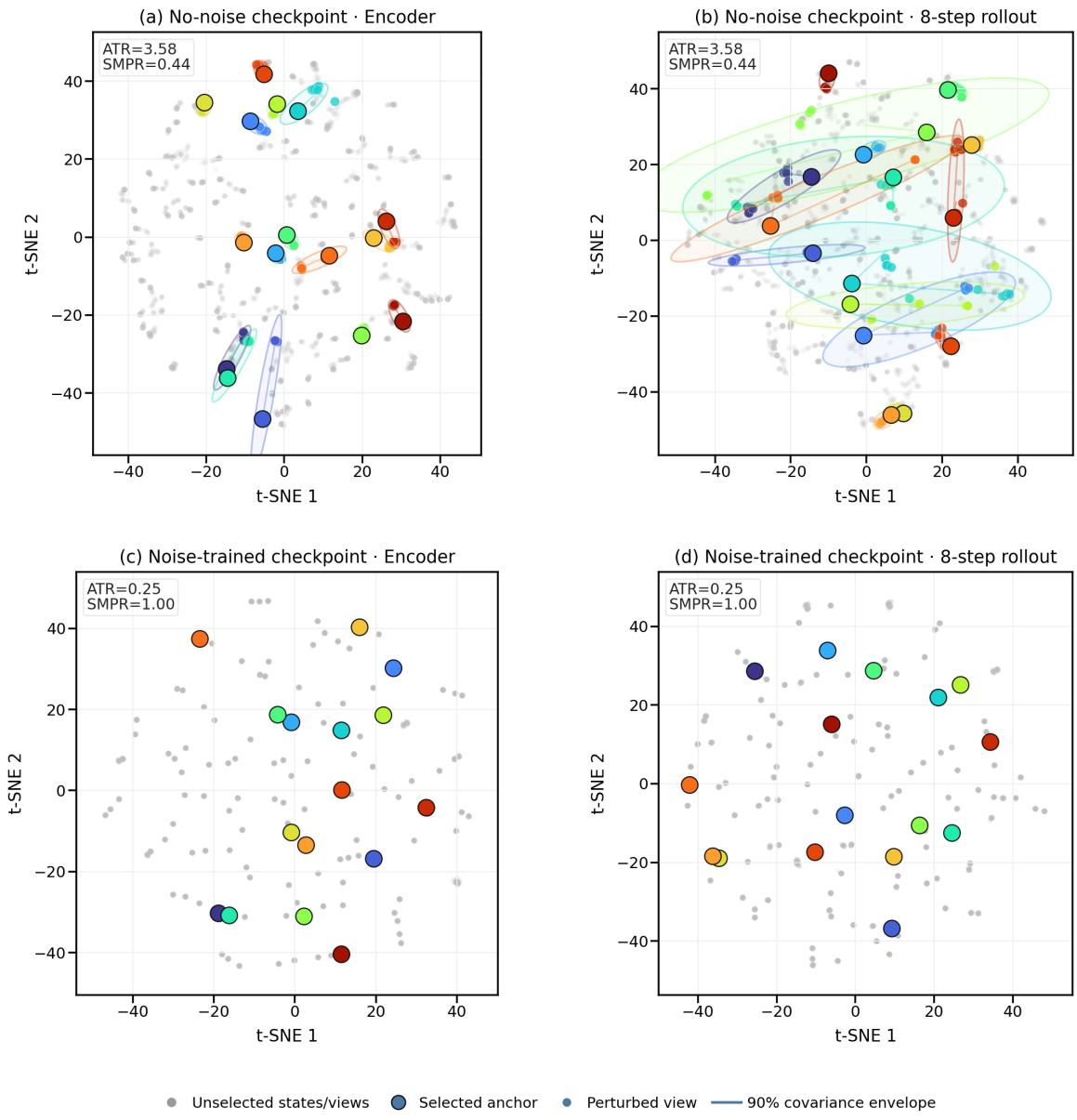

Figure 7: Qualitative PushT neighborhoods for matched perturbations at the unaugmented and $\sigma_{\max} = 0.08$ checkpoints, shown in encoder and eight-step rollout spaces. t-SNE is not metric preserving: no displayed distance, area, or overlap enters a claim; ATR and SMPR are computed in the original space.